

The Yelp Open Dataset contains a large number of fine-grained business categories. To simplify the classification task and improve class balance, these categories were manually mapped into seven broader business categories (Restaurants, Retail, Health, Services, Entertainment, Hospitality, and Education). The mapping was developed based on the Yelp category hierarchy and domain knowledge, with the goal of creating meaningful groups for security risk analysis.

The sampled_df starts with these columns from the merged data:

#	Column Name	Type	Description
1	review_id	string	Unique review ID
2	user_id	string	User who wrote review
3	business_id	string	Business ID
4	stars	int	Star rating (1-5)
5	useful	int	Useful votes
6	funny	int	Funny votes
7	cool	int	Cool votes
8	text	string	Review text
9	date	datetime	Review date
10	main_category	string	Business category (target)
11	is_open	int	1=open, 0=closed
12	latitude	float	Business latitude
13	longitude	float	Business longitude
14	city	string	City name

Text Features

#	Column Name	Type	Description
16	clean_review	string	Cleaned review text (lowercase, no punctuation)

Time Features

#	Column Name	Type	Description
17	review_date	datetime	Review date (converted)
18	review_hour	int	Hour of day (0-23)
19	review_day_of_week	int	Day of week (0=Mon, 6=Sun)
20	is_night	int	1 if 9PM-6AM, else 0
21	is_weekend	int	1 if Sat/Sun, else 0

Review Characteristics

#	Column Name	Type	Description
22	review_length	int	Number of characters
23	word_count	int	Number of words
24	avg_word_length	float	Average word length

Sentiment Features

#	Column Name	Type	Description
25	sentiment_compound	float	Overall sentiment (-1 to +1)
26	sentiment_neg	float	Negative sentiment score (0-1)
27	sentiment_pos	float	Positive sentiment score (0-1)
28	sentiment_neu	float	Neutral sentiment score (0-1)

Urgency Features

#	Column Name	Type	Description
29	exclamation_count	int	Number of ! marks
30	question_mark_count	int	Number of ? marks
31	capitalized_words	int	Number of ALL CAPS words
32	urgency_score	int	Exclamation + (CAPS × 2)

Security Category Features

#	Column Name	Type	Description
33	security_crime	int	Crime-related word count
34	security_safety	int	Safety-related word count
35	security_personnel	int	Security personnel word count
36	security_incidents	int	Incident-related word count
37	security_environment	int	Environmental risk word count
38	total_security_keywords	int	Sum of all security categories

Business Context Features

#	Column Name	Type	Description
39	is_open	int	1=open, 0=closed (filled)
40	recent_review_count	int	Reviews in last 30 days
41	business_density	int	Number of businesses in city

Tip Features (Added after merging tip_features)

#	Column Name	Type	Description
42	tip_count	int	Number of tips for business
43	avg_tip_sentiment	float	Average sentiment of tips
44	std_tip_sentiment	float	Standard deviation of tip sentiment
45	min_tip_sentiment	float	Minimum (most negative) tip sentiment
46	max_tip_sentiment	float	Maximum (most positive) tip sentiment
47	total_tip_security_keywords	int	Total security keywords in tips
48	avg_tip_security_keywords	float	Average security keywords per tip
49	max_tip_security_keywords	int	Maximum security keywords in a tip
50	avg_tip_age_days	float	Average age of tips in days
51	newest_tip_age_days	int	Age of newest tip in days
52	security_tip_ratio	float	Security tips / Total tips
53	security_volatility	float	$\text{std_tip_sentiment} \times \text{security_tip_ratio}$

1. X_text (Review Text)

Variable	Type	Shape	Description
X_text	pandas Series	(50,000,)	Original review text (before vectorization)
X_train_text	pandas Series	(40,000,)	Training review text
X_test_text	pandas Series	(10,000,)	Test review text
X_train_tfidf	scipy.sparse.csr_matrix	(40,000, 5,000)	TF-IDF features for training
X_test_tfidf	scipy.sparse.csr_matrix	(10,000, 5,000)	TF-IDF features for test

2. X_base_meta (12 Base Security Features)

Variable	Type	Shape	Description
X_base_meta	numpy.ndarray	(50,000, 12)	12 base features
X_train_base	numpy.ndarray	(40,000, 12)	Training base features
X_test_base	numpy.ndarray	(10,000, 12)	Test base features
X_train_base_scaled	numpy.ndarray	(40,000, 12)	Scaled training base features
X_test_base_scaled	numpy.ndarray	(10,000, 12)	Scaled test base features

3. X_category_meta (5 Security Category Features)

Variable	Type	Shape	Description
X_category_meta	numpy.ndarray	(50,000, 5)	5 category features
X_train_cat	numpy.ndarray	(40,000, 5)	Training category features
X_test_cat	numpy.ndarray	(10,000, 5)	Test category features
X_train_cat_scaled	numpy.ndarray	(40,000, 5)	Scaled training category features
X_test_cat_scaled	numpy.ndarray	(10,000, 5)	Scaled test category features

4. X_tip (12 Tip Features)

Variable	Type	Shape	Description
X_tip	numpy.ndarray	(50,000, 12)	12 tip features
X_train_tip	numpy.ndarray	(40,000, 12)	Training tip features
X_test_tip	numpy.ndarray	(10,000, 12)	Test tip features
X_train_tip_scaled	numpy.ndarray	(40,000, 12)	Scaled training tip features
X_test_tip_scaled	numpy.ndarray	(10,000, 12)	Scaled test tip features

Random Forest outputs a **prediction** (category or number).

Output	Type	Description
Category Prediction	String	"Restaurants", "Retail", "Health", etc.
Probability	Float (0-1)	Confidence in the prediction
Decision Path	List	Which trees voted for which category

1.Input: Feature matrix (rows = reviews, columns = features)

2.Output: Category prediction (e.g., "Restaurants")

3.Process: 100 trees vote → majority wins

4.In Our Project: Used for feature importance analysis

5.Why: Tells us which features are most predictive

INPUT: "The restaurant was great but the parking lot was dark"



1. Extract Features (29+ features)

- Clean text
- Time features (hour, night)
- Sentiment (-0.58)
- Security keywords (safety: 1, environment: 2)
- Business context



2. Transform features

- TF-IDF (5,000 text features)
- Scale numeric features



3. Fix Feature Mismatch

- Expected: 5,029 features
- Actual: 5,017 features
- Problem: Single review has fewer words
- Solution: Add 12 zeros (padding)



4. Make Prediction

- Best model predicts category
- Calculate confidence



OUTPUT: {

'category': 'Restaurants',

'confidence': 94.25%,

'risk_level': '  High',

'risk_score': 5.0

}

1	Extract features	Match training data
2	Transform features	Same as training pipeline
3	Align feature count	Single review has fewer words
4	Make prediction	Use best trained model
5	Calculate risk	Score security keywords